

Assignment Summary – Delta Lake MERGE Implementation

Overview

For this assignment, I worked with the **Superstore sales dataset**, which contains **9,994 order records** along with customer, product, sales, shipping, and profit information. The objective was to perform incremental data processing using **Delta Lake** in Databricks by loading the dataset, cleaning it, simulating new incoming data, and applying **MERGE INTO** operations. In addition, I implemented **Slowly Changing Dimension (SCD Type 2)** to preserve historical data.

What I Did

I started by uploading the master and incremental CSV files into my Databricks workspace and loading the master dataset into a Spark DataFrame. While loading the data, I enabled automatic schema inference and verified that the dataset was read correctly.

One issue I encountered was that Delta Lake does not handle column names with spaces and special characters well. The original dataset contained columns such as **"Row ID"**, **"Ship Mode"**, and **"Sub-Category"**, which caused errors while creating Delta tables. To resolve this, I standardized all column names by replacing spaces, hyphens, and special characters with underscores (for example, **Row_ID**, **Ship_Mode**, and **Sub_Category**). This created a consistent schema throughout the project.

After preparing the schema, I stored the dataset as a Delta table named **superstore_raw**.

Data Cleaning

Before applying any transformations, I checked the dataset for duplicate records and missing values. The original Superstore dataset was already clean, with no duplicate rows or null values. However, instead of skipping the cleaning step, I created a small synthetic dataset containing duplicate rows and missing values to demonstrate that the cleaning logic worked correctly.

I removed duplicate records using `dropDuplicates()` and handled missing values using `fillna()`. After verifying the results, I applied the same cleaning process to the original dataset and saved it as the **superstore_master** Delta table.

Incremental Data Processing

Next, I loaded the incremental dataset, which contained a combination of updated records and new records. Similar to the master dataset, I standardized the column names and stored it as another Delta table named **superstore_incremental**.

This incremental dataset was used to simulate a real-world scenario where new data continuously arrives and existing records need to be updated.

MERGE Operation

The main objective of this assignment was to perform a Delta Lake **MERGE** operation.

I used **Row_ID** as the primary key to compare the master and incremental datasets. During the merge:

- Existing records were updated with the latest values.
- New records that were not present in the master table were inserted automatically.

This demonstrated how Delta Lake simplifies incremental data processing by combining update and insert operations into a single statement.

Implementing SCD Type 2

To preserve historical information, I implemented **Slowly Changing Dimension (SCD Type 2)**.

Instead of replacing existing records, I added three additional columns:

- **effective_date**
- **end_date**
- **is_current**

Whenever a record changed, the previous version was marked as inactive by updating the **end_date** and setting **is_current** to **False**. A new version of the record was then inserted with the latest values and **is_current = True**.

This approach maintains the complete history of changes rather than keeping only the latest version of each record.

Challenges Faced and How I Solved Them

During implementation, I faced several practical challenges while working in Databricks.

The first issue was related to **invalid column names**. Delta Lake raised errors because the dataset contained spaces and special characters in the column names. I resolved this by renaming all columns using underscores before creating the Delta tables.

Another issue occurred during the **MERGE** operation. Initially, the master and incremental tables had inconsistent schemas because one table still contained the original column names while the other used renamed columns. To solve this, I recreated both Delta tables using the standardized schema and enabled **overwriteSchema** to ensure consistency.

I also encountered issues where previously created Delta tables retained the old schema. Dropping and recreating the tables before writing the updated DataFrames resolved these conflicts.

Working through these problems helped me better understand how Delta Lake handles schemas and why maintaining consistent column names is important when performing merge operations.

Validation

After completing the merge process, I verified the results by:

- Checking the total number of rows in the final Delta table.
- Ensuring that every **Row_ID** was unique.
- Verifying that updated records reflected the latest values.
- Confirming that newly inserted records were present.
- Reviewing the **DESCRIBE HISTORY** output to validate the Delta transaction history.

These checks confirmed that the merge operation was completed successfully without introducing duplicate records.

What I Learned

This assignment provided practical experience with Delta Lake and Databricks. I learned how to load datasets into Delta tables, perform data cleaning, create incremental data pipelines, and implement **MERGE INTO** operations for efficient upsert functionality.

More importantly, I gained hands-on experience in handling schema-related issues, standardizing column names, recreating Delta tables when schemas change, and implementing **SCD Type 2** for maintaining historical records.

Overall, this assignment helped me understand how Delta Lake supports reliable and scalable data engineering workflows, especially for real-world incremental data processing scenarios.